

C++ STL 容器与算法可视化学习平台 (STL Visual Lab)

小组：助教分配28组

组长：2400013198 覃耀纬 (刘家瑛班)

组员：2500012394 叶子成 (郭炜班) 2500012917 曹梓钊 (刘家瑛班)

2026年7月

目录

1	程序功能介绍	2
1.1	项目背景与目标	2
1.2	核心功能	2
2	项目各模块与类设计细节	3
2.1	整体架构设计	3
2.2	核心类设计	4
2.2.1	VisualBase (抽象可视化基类)	4
2.2.2	VisualVector & VisualList (数据结构演示类)	4
2.2.3	VisualPermutation (特殊算法演示类)	4
2.2.4	QuizDatabase (测验题数据库类与智能错题推荐)	5
2.2.5	SubQuizWidget (安全多线程沙盒评测类)	5
3	小组成员分工情况	6
4	项目总结与反思	6
4.1	项目亮点	6
4.2	开发过程中的问题	6
4.3	收获与反思	7
4.4	其他说明	7

1 程序功能介绍

1.1 项目背景与目标

C++ 标准模板库 (STL) 是 C++ 程序设计的核心基石，然而其底层的内存模型（如 `std::vector` 的动态倍增扩容、`std::list` 的双向链表指针重连）以及算法执行流程（如 `std::sort` 的内省排序、`std::next_permutation` 的字典序排列生成）对于初学者来说抽象且难以直观理解。

本项目基于 Qt 6.11 (C++17) 框架开发了一套交互式 C++ 容器与算法可视化学习平台 (STL Visual Lab)。通过实时动画演示、单步调试跟踪、代码高亮对照以及智能错题推荐评测系统，帮助初学者直观掌握 STL 底层机理。

1.2 核心功能

1. 主控中心仪表盘：采用现代化高科技深色扁平设计，提供“数据结构”与“STL算法”两大核心板块的直观入口，配备高清图示。
2. 数据结构可视化平台：
 - `std::vector`：动态演示 `push_back`、`insert` 触发的内存申请、元素拷贝及 `capacity` 倍增扩容过程。
 - `std::list`：以图形化节点 and 动态箭头指针展示插入、删除时“双向链表断开与重连”的动画细节。
 - `std::stack & queue`：动态模拟栈的“后进先出 (LIFO)”与队列的“先进先出 (FIFO)”流水线吞吐流程。
3. 算法流程可视化平台：包含 `std::sort`（快速/插入/堆排序选择）、`std::lower_bound`（二分查找区间收缩）、`std::reverse`（双指针交换）、`std::unique`（快慢指针去重覆盖）、`std::merge`（双指针归并有序区间）、`std::remove`（元素前移与逻辑尾部返回）、`std::rotate`（三阶段翻转旋转）与 `std::next_permutation`（字典序全排列生成动画）这 8 种核心算法的可视化。它们是 STL 学习中最典型且效果直观算法。
4. 代码高亮与对照系统：所有演示均配备实时高亮 C++ 源码/伪代码窗口，动画执行的每一步均与代码行严格同步，可以实现动图与代码对照学习。
5. 多功能动画控制器：支持自动播放/暂停、单步前进、单步后退（支持历史状态回溯）、重置动画，并支持一键导出当前画布为 `.png` 格式的图片。便于学生自己根据状况控制并导出以完成整理。
6. 智能测验与本地编译：每个模块集成 15 道专项练习题（10道单选题 + 5道编程题）。编程题支持直接应用内置的编辑器中编写 C++ 代码，通过后台调用本地编译器（MinGW g++）进行实时编译，并运行严苛的单元测试 `assert`，反馈测试结果。系统自动记录用户

错题，并基于 `UserProfile` 错题库在下次测验时进行个性化智能题目推荐。推荐的原则是基于用户上一次错题的顺序，错的次数越多，下一次检测题目所处的位置就在越前面。

2 项目各模块与类设计细节

2.1 整体架构设计

项目严格遵循与面向对象的设计原则，将主窗口控制、底层数据管理以及动画逻辑渲染完全分成不同部分。主要的类及其相互关系说明如下：

1. **MainWindow (主窗口)**: 整个系统的控制中心与导航中枢。其组合了 `QStackedWidget` 控件，通过切换页面来呈现主仪表盘、数据结构可视化界面以及算法可视化界面。同时持有 `UserProfile` 指针以获取和更新用户的测验历史。
2. **VisualBase (抽象可视化基类)**: 定义了所有可视化演示模块的通用接口和行为规范。提供 `stepForward()` (动画前进一步)、`stepBackward()` (动画回退一步)、`reset()` (重置动画) 和 `isAtEnd()` (检查动画是否结束) 等纯虚接口。作为所有可视化展示子类的基类。
3. **具体可视化演示类 (数据结构与算法子类)**:
 - 数据结构类: `VisualVector`、`VisualList`、`VisualStackQueue`。
 - 算法类: `VisualSort`、`VisualLowerBound`、`VisualPermutation`，以及 `VisualReverse`、`VisualUnique`、`VisualMerge`、`VisualRemove`、`VisualRotate` 等。

全部继承自 `VisualBase`，并重写其所有纯虚接口，实现各自特定的单步动画逻辑与基于快照栈的状态回退。

4. **SubQuizWidget (智能小测验组件)**: 负责小测验界面的展示、用户代码的收集、后台多线程编译和单元测试评测。其依赖 `QuizDatabase` 题库类来获取题目，并依赖 `UserProfile` 来记录答错的题目ID以支持错题推荐。
5. **QuizDatabase (题库数据库类)**: 管理系统中所有 165 道单选题和编程题的静态数据。被 `SubQuizWidget` 调用，并能配合 `UserProfile` 提供的错题历史对题目列表进行动态重排，实现错题优先推荐。
6. **UserProfile (用户配置与历史管理类)**: 采用单例模式 (Singleton) 设计，负责将用户的答题历史、错题频次序列化为本地 JSON 文件进行持久化存储。被 `MainWindow` 和 `SubQuizWidget` 获取和更新。

2.2 核心类设计

2.2.1 VisualBase (抽象可视化基类)

所有可视化演示组件的基类，定义了动画驱动的标准接口，并提供了代码高亮等公用辅助函数。

- 主要接口：

- `virtual void stepForward() = 0`: 执行动画的下一步，更新数据状态并触发重绘。
- `virtual void stepBackward() = 0`: 利用内部状态历史栈，回退到动画的上一步。
- `virtual void reset() = 0`: 重置所有动画状态至初始输入。
- `virtual bool isAtEnd() const = 0`: 检查动画是否播放完毕。
- `virtual void exportImage()`: 抓取当前 `paintEvent` 绘制的 `QWidget` 视口，保存为本地图片。

2.2.2 VisualVector & VisualList (数据结构演示类)

- **VisualVector**: 维护一个物理数组和逻辑大小。当 `size > capacity` 时，模拟动态倍增扩容（分配 2 倍内存 → 迁移旧元素 → 释放旧内存 → 插入新元素）的阶段性的重绘。
- **VisualList**: 维护 `ListNode` 节点的几何位置坐标及指针连接线。通过在 `stepForward` 中动态插值更新箭头弧度和节点坐标，平滑呈现链表插入/删除时指针断开重连的视觉效果。

2.2.3 VisualPermutation (特殊算法演示类)

- 实现了 `std::next_permutation` 的全步骤可视化。
- **数据结构**: `struct PermAnimStep` 记录每一步的数组快照、当前高亮元素索引、翻转区间起点以及对应的 C++ 代码行。
- **流程呈现**: 第一阶段从右向左寻找断点 `i`，第二阶段寻找交换数 `j` 并交换，第三阶段翻转 `i+1` 到末尾的子区间。使用不同的色彩高亮区分指针指向和反转范围。

2.2.4 QuizDatabase (测验题数据库类与智能错题推荐)

- **题库数据集：**管理全系统 11 个模块（8 个经典模块 + 3 个新增模块）的测验题库。目前题库总量达到 **165 道题**（110 道多选题 + 55 道编程实践题，具体题目由 ai 根据课件讲义所生成）。所有编程题的 `testCasesCode` 均采用安全的 `R"raw(...)raw"` 进行定界以避免解析冲突。
- **智能错题推荐原理：**
 - **实时记录：**答题过程中如果单选题答错或编程题编译运行失败，`SubQuizWidget` 会实时调用 `UserProfile::recordError()`。
 - **本地持久化：**`UserProfile` 会以 JSON 格式将错题及错误频次序列化保存到本地 `build/user_profile.json` 中。
 - **动态重排推荐：**在每次启动小测验加载题目时，`QuizDatabase::getQuestionsFor` 会获取用户已保存的错题 ID 列表，并执行重排序划分算法，将用户答错过的题目动态移动到题库向量的最前端。通过“错题优先测试”机制实现了轻量且高针对性的个性化推荐复习。

2.2.5 SubQuizWidget (安全多线程沙盒评测类)

- **环境自适应与高可移植性：**评测器抛弃了写死本地路径的做法。系统首先检测自定义环境变量 `STL_GPP_COMPILER`，若为空则调用 `QStandardPaths::findExecutable` 检索系统 `PATH` 中注册保持的 `g++`；最后备用默认的本地 `MinGW` 相对路径。若所有途径均未找到编译器，会向控制台返回清晰的配置向导。
- **系统安全临时目录：**为了避免权限冲突和垃圾文件残留，测试用的代码文件（如 `temp_compile.cpp`）与生成的编译程序均会被写入到系统隔离的安全临时目录 `QStandardPaths::writableLocation` 中，并在评测完毕后自动清理。
- **多线程编译与响应式 UI：**编译和执行属于高 CPU 消耗及阻塞性动作。为了彻底避免主线程卡顿（UI 冻结），`SubQuizWidget` 利用 `QThread::create` 创建并启动独立的后台工作线程。工作线程执行完耗时的编译 and 运行后，利用 `QMetaObject::invokeMethod` 将数据封包安全地投递回主线程更新 UI。
- **沙盒评测安全防范与资源隔离：**
 - **时间限制 (Time Limit)：**编译过程设置 8 秒硬性超时，程序执行阶段设置 2 秒硬性超时，利用进程强行杀除（`kill()`）机制，杜绝用户编写死循环或逻辑死锁代码引发的 CPU 暴涨及线程僵死。
 - **输出限制 (Output Limit)：**在读取执行结果的标准输出（`stdout`）和标准错误输出（`stderr`）时，设定了 64KB 的数据大小截断上限，防止用户通过死循环输出导致系统

内存耗尽崩溃。

- **隔离提示说明：**系统提示用户评测沙盒对文件系统写入及网络访问进行了限制说明，防止用户输入恶意代码。

3 小组成员分工情况

姓名	学号	班级	具体负责的工作与贡献
叶子成	2500012394	郭炜班	负责项目整体框架搭建，开发 <code>MainWindow</code> 核心导航与动画播放控制逻辑，实现 <code>VisualBase</code> 可视化基类及 <code>SubQuizWidget</code> 沙盒评测与多线程评测机制。
曹梓钊	2500012917	刘家瑛班	负责数据结构模块（ <code>VisualVector</code> 、 <code>VisualList</code> 、 <code>VisualStackQueue</code> ）的可视化动效开发，绘制平台所需的全部科技感插图并优化 UI 界面样式。
覃耀纬	2400013198	刘家瑛班	负责算法状态机模块（包含 <code>VisualPermutation</code> 、 <code>VisualRotate</code> 、 <code>VisualRemove</code> 等 8 个算法）的单步执行逻辑编写，以及 <code>QuizDatabase</code> 题库中 165 道题目的测试与校验。

4 项目总结与反思

4.1 项目亮点

1. **深度互动的学习模式：**将抽象的 C++ 内存模型和迭代器操作拆解为纯手工绘制的单步状态机动画，直观性远超文字教材；对照的高亮源码让使用者对底层函数的控制逻辑一目了然。
2. **沙盒评测系统的集成：**无需安装第三方 IDE，直接在应用内实现 C++ 代码的编写、编译和沙盒执行，为学习者提供了闭环的“学-练-评”体验。
3. **流畅的视觉和交互设计：**界面使用现代深色主题，合理搭配了高对比度的霓虹配色（蓝、绿、橙、黄等），并在主页面补齐了精美的高清科技感图示，交互微动效平滑自然。

4.2 开发过程中的问题

1. **中文乱码和文件编码问题：**开始时一小组成员在 Windows 的 Qt 里写代码，但是当把项目发给用 Mac 的同学或者在别人电脑上打开时，代码里的中文注释和全部变成了乱码，编译时还一直报错非法字符。后发现是因为 Windows 默认的 GBK 编码和 macOS/Linux 的 UTF-8 编码冲突。最后小组成员把所有的 C++ 源代码文件都重新用 UTF-8 格式保存，并再项目配置文件里强行指定了 `-utf-8` 的编译参数，才解决这个问题。
2. **C++ Raw String 括号提前闭合的 bug：**小组成员在写小测验题库时，需把很多 C++ 示例代码直接当成字符串存进数据库里。为了省去写转义字符的麻烦，小组成员用了 C++ 的原始字符串字面量 `R"(...)"`。但写发现，如果测试代码里也刚好有括号（比如有一行代码是 `isBalanced("(()))"`），编译器就会误以为测试代码里的右括号就是外层原始

字符串的结束标志，产生大量报错。后经讨论，在 `R` 后面加上自定义的定界符，改写成 `R"raw(...)raw"`，这样编译器就不会错认右括号。

3. **撤销功能实现困难：**小组成员本来想让用户能像看视频一样，可以随时点“上一步”进行撤销。但开始编写代码时才发现，很多 STL 算法的执行是不可逆的（比如 `std::unique` 去重，它是直接把后面的元素往前移并覆盖掉前面的数据，被覆盖的数据就彻底丢了）。如果去写每一个算法的逆向算法，代码实在过于复杂，也容易写出 bug。最后解决办法是：每次动画前进一步，我们就把当前容器里所有元素的数据和位置进行“快照”，丢进轻量级的历史栈里；当用户点“上一步”时，我们就直接把栈顶的快照弹出来重新画一遍，效果较好。
4. **多人用 Git 合作时的代码冲突和同步问题：**三个人分工开发不同的模块，用 Git 进行代码协作。因为大家都是第一次在比较大的项目里合作，开始时出现几个人同时改了同一个文件，然后在 push 的时候就导致冲突。有时合并时会覆盖他人代码，连项目都跑不起来；还有人改了配置文件却未及时交流，导致拉取代码后直接报错。后小组成员写完有了进展赶紧提交，并且经常核对冲突的代码，才逐渐减少错误。

4.3 收获与反思

通过本项目的协作开发，小组成员不仅巩固了 C++ STL 各类容器（`vector` 扩容机制、`list` 指针指向）与典型区间算法（`next_permutation`、`unique`）的底层原理，还深入掌握了 QT 的用法、自定义二维绘图（`QPainter`）以及 C++ 程序的跨平台编译部署技术。大作业也让我们认识到，在软件设计中，前期制定严谨的接口规范（如 `VisualBase` 的纯虚接口）对于后期多人并行协作开发具有重要的保障作用。

4.4 其他说明

1. 由于多种算法和容器的主要界面类似，功能也类似，且工程量大。在开发了头2-3个主要算法和容器后，小组使用了ai对剩下的算法和容器先进行了模板式代码的生成，再通过具体修改实现剩下的算法和容器。同时对应的知识卡片和训练题目由ai根据程序设计实习课件讲义生成。
2. 作业中主界面仪表盘的图片 and 分类导航所使用的配图是由ai生成的，节省了网上寻图或是手绘的时间。